

ACO TSP Solver

Ant Colony Optimization for the Travelling Salesman Problem

Ant System on natID Dense Matrices, with Real-Time Visualization

STUDENT

Armin Memišević · 20016

COURSE

Numerical Optimization

PROFESSOR

Prof. Dr. Izudin Džafić

ACADEMIC YEAR

2025/26



The Core Idea — Why a Metaheuristic, Not Exact Search

Exact search is hopeless even at 25 cities; ACO trades the optimality guarantee for controllable computation.



The TSP is **NP-hard**.



Proposal: **15–25 cities**, all parameters constant during a run.



Basic Ant System, **no local search**: every result traces to one textbook update rule.

WHY NOT EXACT SEARCH?

Brute force / Held–Karp

INTRACTABLE

$$\frac{(n-1)!}{2}$$

distinct tours

4.4×10^{10} at $n = 15$

3.1×10^{23} at $n = 25$

$$O(n^2 2^n)$$

at $n = 25$: $\approx 2 \times 10^{10}$ operations, $\approx 8 \times 10^8$ table entries (several GB)

Ant System / this project

$$m \times O(n^2) \text{ per iteration}$$

Within **0.7%** of the optimum on average (slide 18)

01

SECTION 01

Problem Formulation

The TSP as a minimization problem over two natID dense matrices

AGENDA

- TSP objective & constraint
- Real-town instances and distance model
- Distance & pheromone matrices



TSP Objective & Problem Instance

Real geography, with air distances within $\pm 2\%$ of great-circle distance.

OBJECTIVE

$$\min_{\pi \in S_n} f(\pi) = \sum_{i=1}^{n-1} d_{\pi_i \pi_{i+1}} + d_{\pi_n \pi_1}$$

s.t. each city appears exactly once

DISTANCE MODEL (λ longitude, ϕ latitude, degrees)

$$d_{ij} = \sqrt{(80(\lambda_i - \lambda_j))^2 + (111(\phi_i - \phi_j))^2} \text{ km}$$



99 real towns (Bosnia.xml)



n drawn uniformly from 15–25



$80 \approx 111.3 \cdot \cos 44.2^\circ$

VS. HAVERSINE (GREAT-CIRCLE)

4,755 pairs > 20 km

−1.8% to +2.0%

mean |error| 0.4% · max 4.7 km



Distance & Pheromone Matrices (natID)

All three matrices are `dense::DblMatrix`, and evaporation and deposit act on them directly.

`dense::DblMatrix`

`_dist`

0	d	d	d	d
d	0	d	d	d
d	d	0	d	d
d	d	d	0	d
d	d	d	d	0

upper → mirrored to lower

symmetric · zero diagonal · upper triangle filled
and mirrored

`dense::DblMatrix`

`_pheromone`

0	τ_0	τ_0	τ_0	τ_0
τ_0	0	τ_0	τ_0	τ_0
τ_0	τ_0	0	τ_0	τ_0
τ_0	τ_0	τ_0	0	τ_0
τ_0	τ_0	τ_0	τ_0	0

$\tau_0 = 1.0$ off-diagonal · diagonal 0

`dense::DblMatrix`

`_deltaPheromone`

0	$\Delta\tau$	0	0	$\Delta\tau$
$\Delta\tau$	0	0	$\Delta\tau$	0
0	0	0	$\Delta\tau$	$\Delta\tau$
0	$\Delta\tau$	$\Delta\tau$	0	0
$\Delta\tau$	0	$\Delta\tau$	0	0

schematic: one tour 1→2→4→3→5→1

zeroed each iteration · holds $\Delta\tau$

`reserve(n, n, nullptr, true) → zeros() → fill`

02

SECTION 02

The Ant System

Probabilistic construction + pheromone feedback, nothing else

AGENDA

- Tour construction (roulette wheel)
- Pheromone update as matrix operations
- The full loop & feasibility guard
- Baseline & convergence indicators



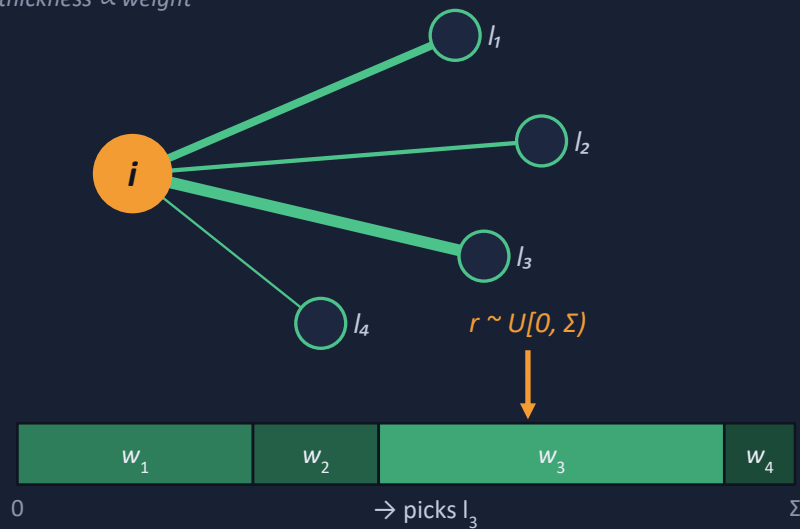
Tour Construction — Roulette Wheel

Each step picks the next city in proportion to trail strength and inverse distance.

$$p_{ij}^k = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{l \in \mathcal{N}_i^k} \tau_{il}^\alpha \eta_{il}^\beta}, \quad \eta_{ij} = 1/d_{ij}$$

SCHEMATIC

thickness $\propto$ weight



MapModel.h

```

1  double tauVal = pher(current - 1, c - 1);
2  double d = dist(current - 1, c - 1);
3  double eta = (d > 1e-6) ? (1.0 / d) : 1e6;
4  double w = std::pow(tauVal, _options.alpha) * std::pow(eta, _options.beta);
5  // ...
6  std::uniform_real_distribution<double> pick(0.0, sum);
7  double r = pick(rng);
8  double acc = 0;
9  next = candidates.back();
10 for (size_t k = 0; k < candidates.size(); ++k)
11 {
12     acc += weights[k];
13     if (r <= acc)
14     {
15         next = candidates[k];
16         break;
17     }
18 }

```



Pheromone Update — Three Matrix Operations

The textbook Ant System rule, mapped one-to-one onto natID operators.

1

$$\tau \leftarrow (1 - \rho) \tau$$

evaporatePheromone()

`_pheromone *= keep`

2

$$\Delta\tau_{ij} = \sum_k Q/L_k$$

beginPheromoneDeposit()
+ depositPheromone()

`zeros(), then per-edge += Q/L`

3

$$\tau \leftarrow \tau + \Delta\tau$$

applyPheromoneDeposit()

`_pheromone += _deltaPheromone`



MapModel.h

```
1 evaporatePheromone();
2
3 beginPheromoneDeposit();
4 for (int a = 0; a < _options.numAnts; ++a)
5     depositPheromone(antTours[a], antLengths[a]);
6 applyPheromoneDeposit();
```



Evaporate the old trail first



Accumulate, then apply — no ant sees another's deposit



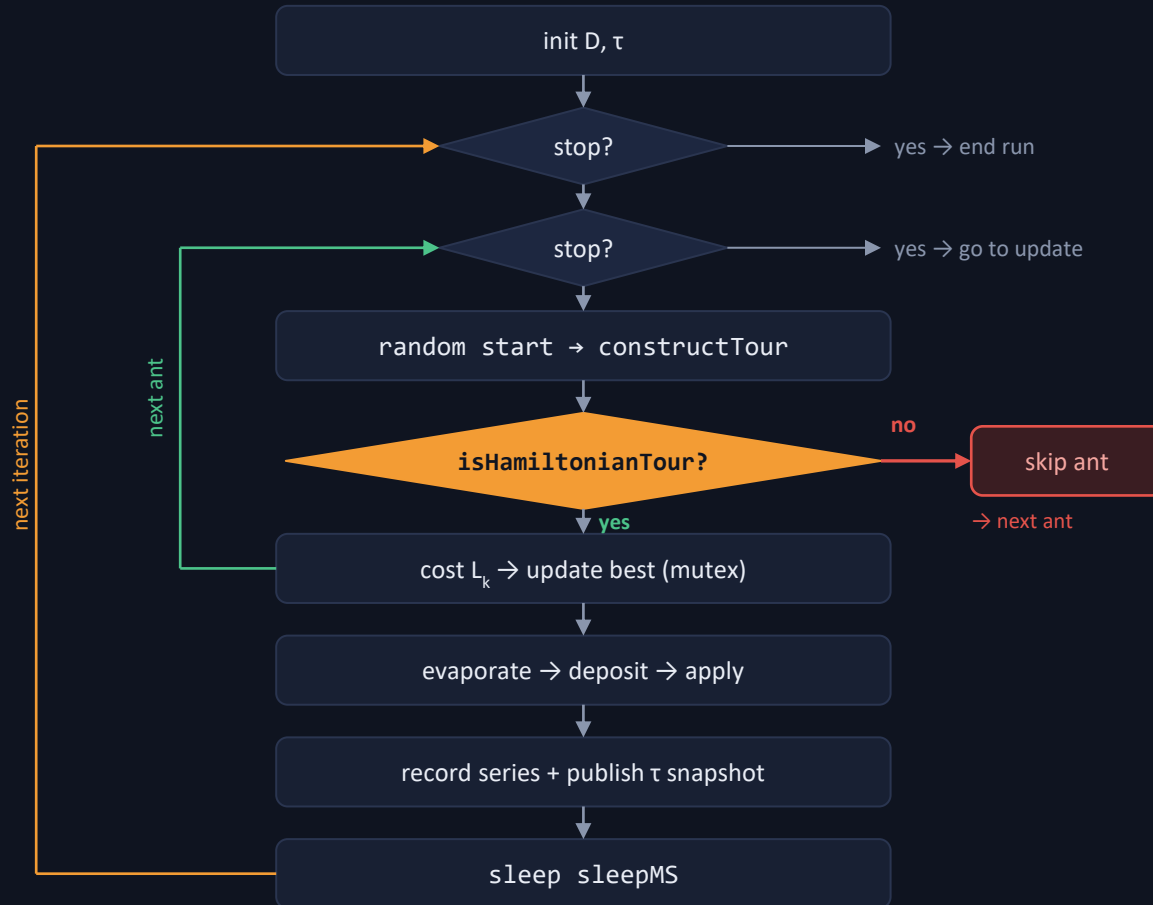
Symmetric, including the closing edge



The Complete Loop & Feasibility Guard

Every tour is checked before it can count.

Model::runACO()



MapModel.h

```

1  bool isHamiltonianTour(const std::vector<GraphType>& tour) const
2  {
3      size_t n = _towns.size();
4      if (n < 2 || tour.size() != n)
5          return false;
6
7      std::vector<bool> seen(n + 1, false); // 1-based ids
8      for (GraphType id : tour)
9      {
10         if (id < 1 || id > (GraphType)n)
11             return false;
12         if (seen[id])
13             return false;
14         seen[id] = true;
15     }
16     return true;
17 }

```

Checked twice: before costing, and again inside depositPheromone.



Nearest-Neighbour Baseline & Convergence Indicators

A best-so-far curve alone proves little; the baseline and iteration-best do.



Best-so-far

monotone



Iteration best

noisy, settles toward best-so-far



Pheromone L1 norm

exported only

Repetitive nearest neighbour

Greedy tour from *every* start city, shortest kept · $O(n^3)$, once per instance · **never seeds the colony**

$$\text{improvement} = 100 \cdot \frac{L_{NN} - L^*}{L_{NN}} \%$$

signed · unclamped

⚠ The L1-norm trap

Every tour enters and leaves each city once $\Rightarrow$ all column sums stay equal.
It measures trail mass, not concentration.

$$\|\tau^{(t)}\|_1 = (1 - \rho) \|\tau^{(t-1)}\|_1 + 2 \sum_k Q/L_k$$

verified to nine decimal places

03

SECTION 03

Real-Time GUI Application

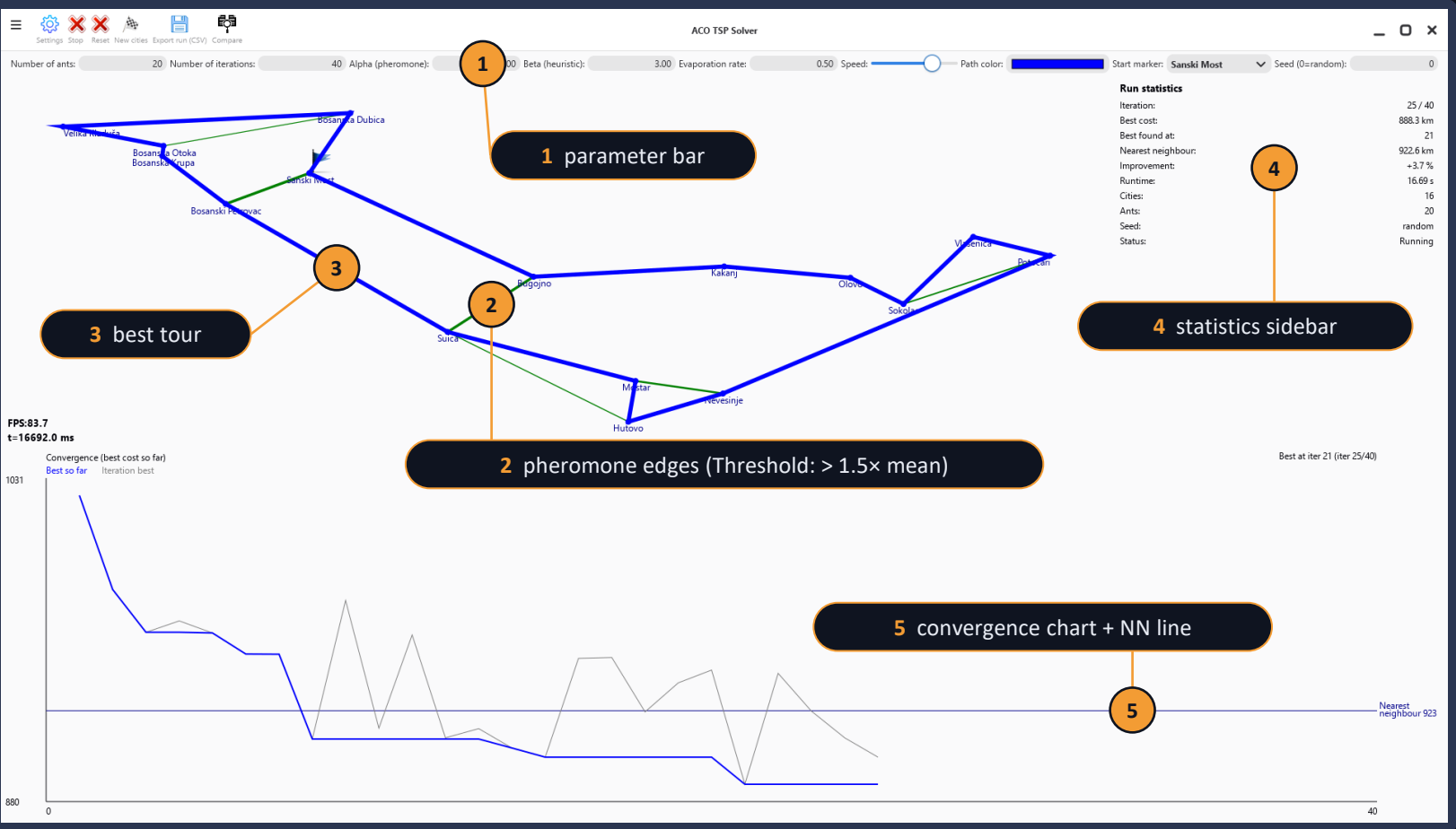
A natID desktop app for watching and measuring the colony

AGENDA

- Interface overview
- Threading model
- Experiments: parameters, compare, export

Interface Overview

Map, pheromone trail, chart and statistics all update live.



Map canvas
animation loop requests 60 fps

Chart & sidebar
request 10 fps, drawn from thread-safe snapshots

Edge thickness
0.5–4.0 px, scaled to trail

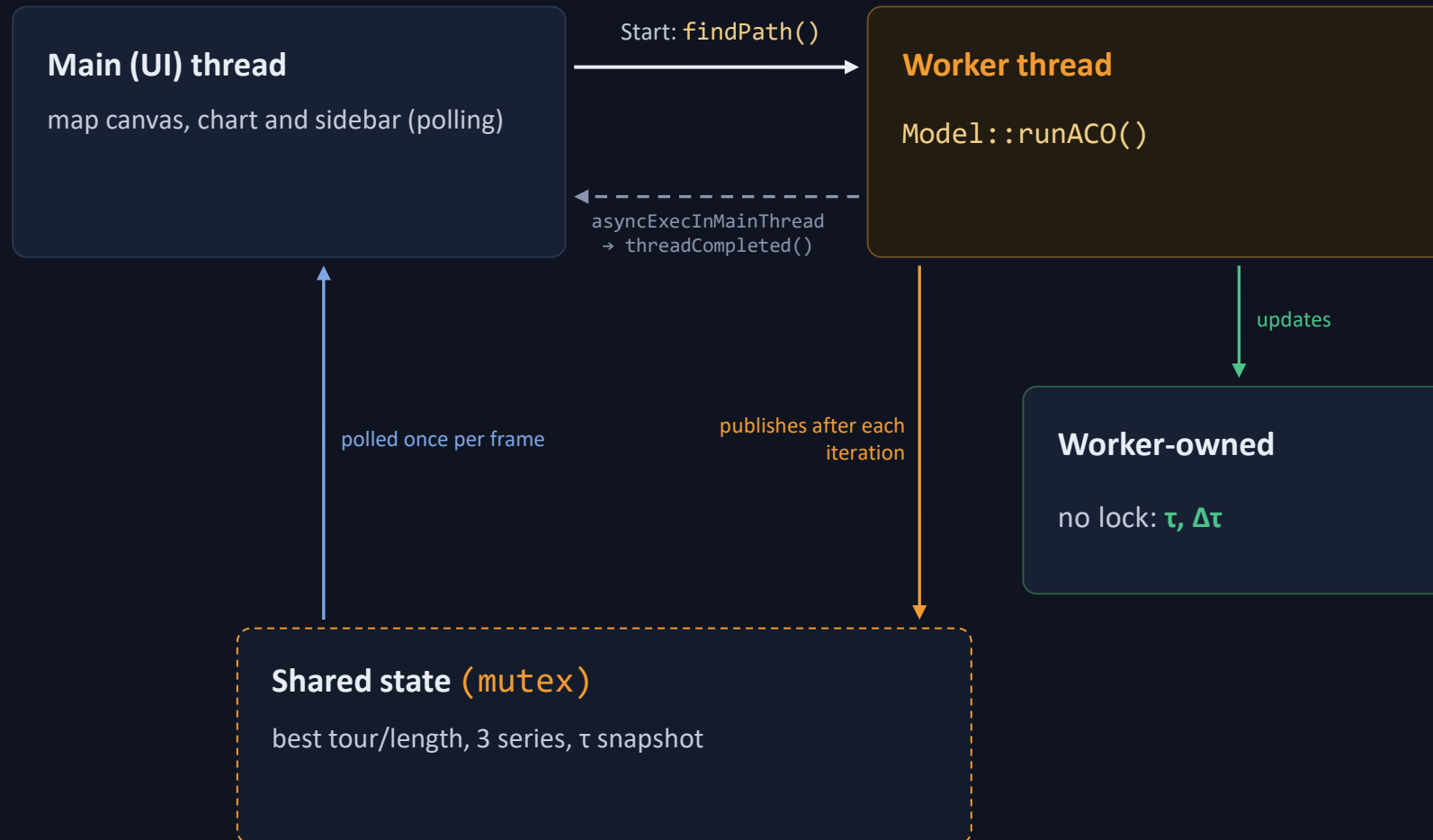
Grid: parameter bar · map (15/20 cols) + sidebar (5/20) · full-width chart

16 cities, 20 ants, 40 iterations, $\alpha=1$, $\beta=3$, $\rho=0.5$, at iteration 25: best tour 888.3 km (found at iter 21), 3.7% shorter than NN 922.6 km.



Threading Model

The worker owns the matrices; the UI only ever sees complete snapshots.



LEGEND



`std::atomic` flags & counters



`gui::Shape` main-thread only
(`_tourShapeDirty`)



Stop = flag, checked per iteration & per ant



Built for Experiments — Parameters, Compare, Export

Change one parameter, rerun on the same cities, compare with numbers.

Comparison of the last two completed runs			
Metric	Newer run	Older run	Difference
Problem			
Cities:	16	16	--
Nearest neighbour:	884.8 km	884.8 km	--
Parameters			
Ants:	20	20	--
Number of iterations:	40	40	--
Alpha (pheromone):	1.00	1.00	--
Beta (heuristic):	5.00	3.00	--
Evaporation rate:	0.50	0.50	--
Seed:	random	random	--
Result			
Best cost:	863.5 km	863.5 km	+0.0 km
Improvement:	+2.4 %	+2.4 %	+0.0 %
Best found at:	6	4	+2
Converged:	yes	yes	--
Final iteration best:	863.5 km	863.5 km	+0.0 km
Mean iteration best:	874.8 km	871.1 km	+3.7 km
Runtime:	0.01 s	0.01 s	-0.00 s

Both runs were solved over the same set of cities.

Same 16 cities, only β differs (5 vs 3): both reach 863.5 km (2.4% below NN); $\beta=5$ finds it 2 iterations later, mean iteration best 3.7 km higher.

Parameter	Symbol	Default	Range
Ants	m	20	1–500
Iterations	N_{iter}	100	1–100,000
Alpha	α	1.0	0–10
Beta	β	3.0	0–10
Evaporation	ρ	0.5	0.01–0.99
Q	Q	100	fixed
τ_0	τ_0	1.0	fixed
Seed	—	$\emptyset$ = random	—

↓ CSV: convergence series + full instance (cities, tour legs, $n \times n$ distance matrix)

Reproducible run = seed + city_draw_index

04

SECTION 04

Verification & Results

Auditing the implementation and checking it against exact optima

AGENDA

- Control-flow audit — "no cheating"
- Mathematical verification
- Independent numerical check



Control-Flow Audit — "No Cheating"

One control path from Start to every reported tour, with no bypass.



control flow (condensed)

```
1 MainWindow::onActionItem (Start)
2   -> MainView::startStop -> readACOParamsFromUI
3   -> ViewMap::start -> Model::findPath -> std::thread
4       -> Model::runACO
5           -> buildDistanceMatrix, initPheromone
6           -> for iter: for ant:
7               constructTour -> isHamiltonianTour -> tourLength
8           -> evaporatePheromone -> beginPheromoneDeposit
9           -> depositPheromone x m -> applyPheromoneDeposit
10          -> getL1Norm, publishPheromoneSnapshot
11          -> asyncExecInMainThread(ViewMap::threadCompleted)
```

WHAT 'NO CHEATING' MEANS

- ✓ No local search (no 2-opt / 3-opt / Or-opt)
- ✓ No precomputed solutions
- ✓ The baseline never seeds the colony
- ✓ No infeasible tours reported or reinforced
- ✓ No elitism, no $\tau_{\min}/\tau_{\max}$ limits



Mathematical Verification

The src/ and res/ trees were cross-checked against the proposal and against Dorigo & Stützle.



Transition rule

$\tau^\alpha \eta^\beta$ over unvisited cities, sampled in proportion to weight



Update rule

evaporate, accumulate Q/L_k from feasible ants, one addition; symmetric, including the closing edge



Objective

tourLength includes the return edge; the exported per-leg column sums to the reported cost



Parameters constant

written only on the UI thread when no run is active, only read by the worker



Matrices

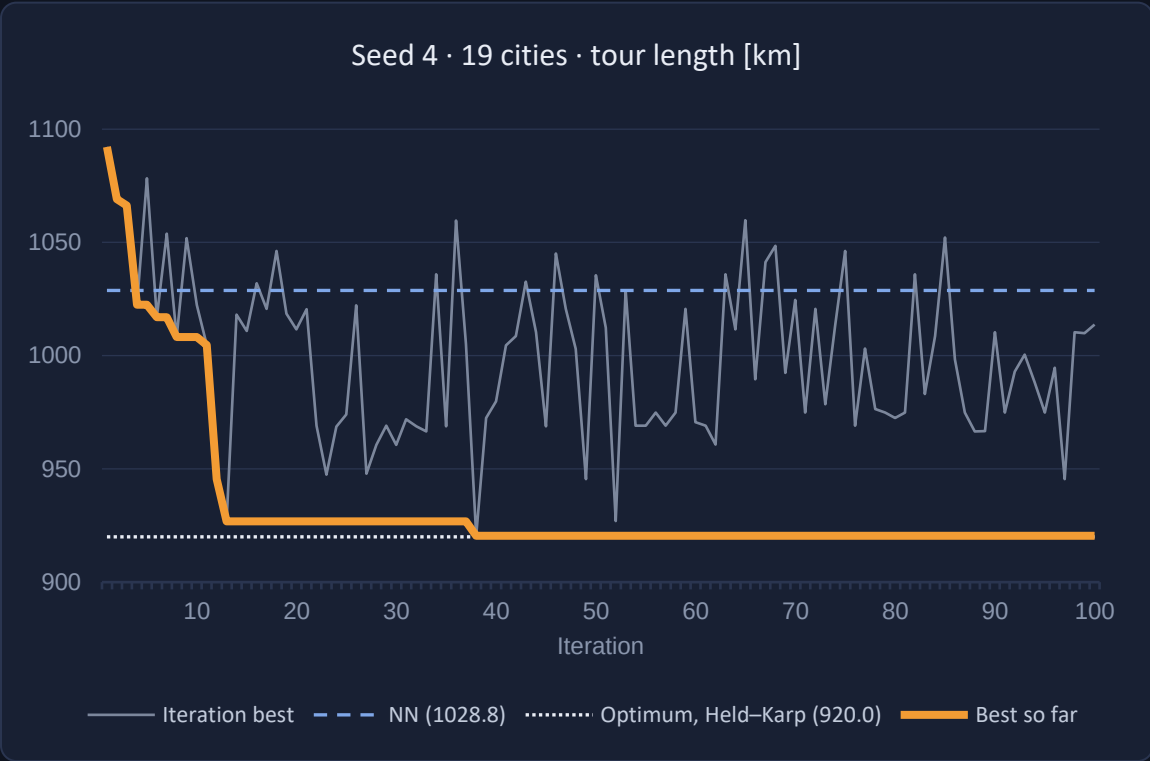
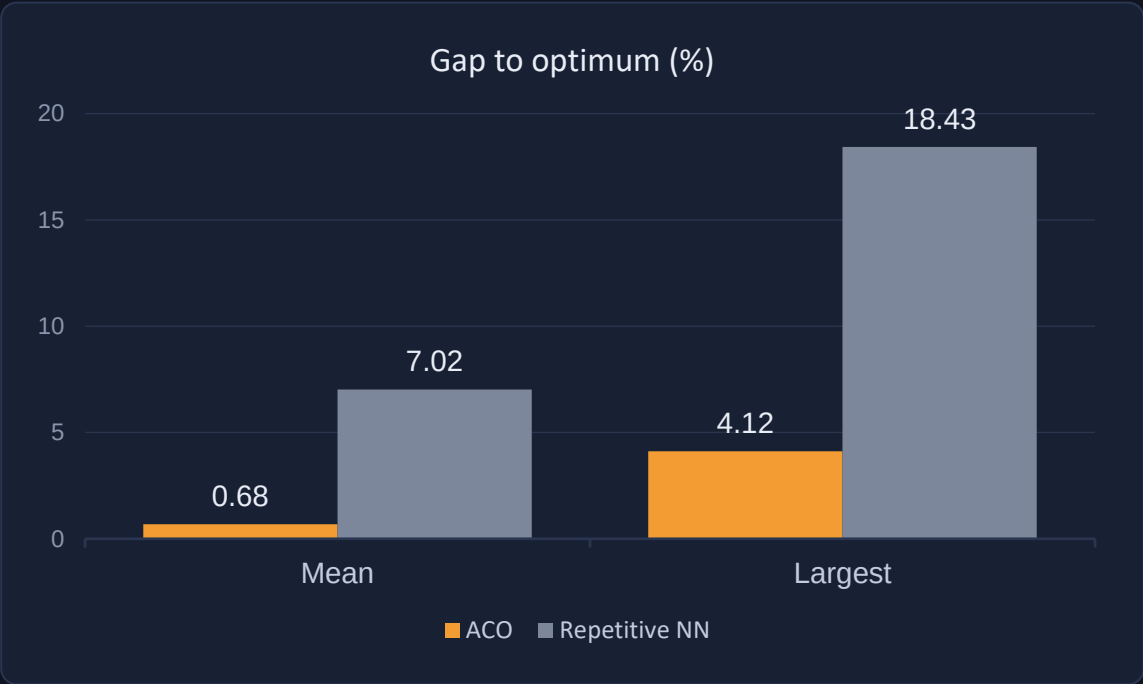
$D, \tau, \Delta\tau$ are **dense::DblMatrix**; evaporation and apply are library operators



Independent Numerical Check — Results

Within 0.68% of the true optimum on average, and never worse than the greedy baseline.

SETUP Faithful C++ re-implementation (same RNG streams, same float/double types) · real Bosnia.xml · defaults $m=20$, 100 iter, $\alpha=1$, $\beta=3$, $\rho=0.5$, $Q=100$, $\tau_0=1$ · 30 instances (seeds 1–30) · Held–Karp optimum for $n \leq 20$ (18 instances)



10 / 18
Optimum found

+6.0%
Mean improvement over NN (all 30)

+0.0% to +16.4%
Range

05

SECTION 05

Features, Limitations & Conclusions

What shipped, how it maps to the proposal, and what it taught me

AGENDA

- Proposal vs. delivered
- Tech stack & known limitations
- Lessons learned & future work



Proposal vs. Delivered

Every proposal item delivered, plus additions.

PROPOSED

- ✓ 15–25 random cities
- ✓ $N \times N$ distance and pheromone matrices as natID dense matrices
- ✓ roulette-wheel construction with α , β
- ✓ evaporation + deposit $\propto 1/L$
- ✓ constant parameters
- ✓ background thread from B_S03_Maps
- ✓ live convergence curve
- ✓ map with best tour and pheromone edge thickness
- ✓ sidebar with iteration, best cost, runtime
- ✓ Start/Stop/Reset

ADDITIONALLY DELIVERED

- + Hamiltonian feasibility check
- + separate $\Delta\tau$ matrix
- + per-iteration best curve + nearest-neighbour baseline
- + All/Threshold pheromone modes
- + seed reproducibility
- + CSV export (series + full instance)
- + comparison of the last two completed runs
- + EN/BA UI, Settings, Help, sounds
- + GitHub Actions builds for Windows, macOS (Apple Silicon + Intel), Linux

✓ **One substantive deviation:** real towns instead of synthetic points (lengths in km)



Tech Stack & Known Limitations

Every number in the app is stated for what it is.

BUILT WITH

C++

CMake 3.18+

natID SDK (MatrixLib, natGUI, mainUtils)

dense::DblMatrix

GitHub Actions + SetupCollector

Windows

macOS

Linux

No dependency outside the natID SDK.

KNOWN LIMITATIONS

Reproducibility is per toolchain

std distributions are implementation-defined; the instance CSV compensates

Runtime is wall-clock

includes the animation delay (500 ms default $\Rightarrow$ 100 iterations $\approx$ 50 s)

pheromone₁₁ $\neq$ concentration

it records trail mass per city

Stopping mid-iteration

still applies that update; stopped runs are not compared

Fixed $\tau_0 = 1$ instead of m/C^{nn}

initial trail $< 1\%$ after 7 iterations at $\rho = 0.5$

Air distances

not road distances



Lessons Learned & Future Work

Two lessons I defend hardest: the feasibility guard and the L1-norm insight.



LESSONS

- 01 A best-so-far curve alone proves little
- 02 A solver must check its own constraints**
- 03 Instrumentation must be checked against the math (L1 norm)**
- 04 Threading discipline is part of correctness
- 05 Reproducibility needs more than a seed

➤ FUTURE WORK

- 01 ACS and MAX–MIN AS selectable at runtime
- 02 2-opt local search
- 03 A real concentration measure (row entropy / share of trail on the best tour's edges)
- 04 Platform-independent sampling on mt19937
- 05 TSPLIB instances with known optima

Thank You

Every tour shown is the direct output of a live Ant System run over `natID` dense matrices; the algorithm was verified against exact optima.



Download for
Windows



Download for
Linux



Download for
macOS Intel



Download for
macOS Silicon

github.com/arminn2206/ProjNO_TSPACO_Memisevic

